# Viva Preparation

*Pitches, forty likely questions with honest answers, and a demonstration script*

| | |
|---|---|
| **Project title** | IntelliMetrics AI: A Governed Semantic Layer with TARA, an AI Assistant for Trusted Business Metrics |
| **Document** | Viva Preparation |
| **Team members** | Devi Priya, Priyanka, Akshay and Bhaskar |
| **Institution** | KL University |
| **Academic year** | 2026-27 |
| **Date** | 26-Oct-2026 (IST) |
| **Status** | University project: academic capstone with synthetic demonstration data |

## Table of contents


- 1 How to use this document
- 2 The 30-second pitch
- 3 The 2-minute pitch
- 4 Fact sheet
- 5 Forty likely questions
    - 5.1 About the idea
    - 5.2 About AI and TARA
    - 5.3 About the formula language and security
    - 5.4 About conflict detection and governance
    - 5.5 About data and Indian specifics
    - 5.6 About engineering and results
- 6 Demonstration script (8 minutes)

# 1 How to use this document

It contains two spoken pitches, a one-page fact sheet, forty questions an examiner could ask, and a script for the live demonstration. All numbers come from the measured data files, so they match the Testing Report. Where the honest answer is “we did not do that”, the answer says so; examiners respect that far more than a bluff.

# 2 The 30-second pitch

> “In most companies, the same word, such as *revenue*, means different numbers to Sales, Finance and Marketing, and nobody notices until two reports disagree. IntelliMetrics AI keeps one governed definition for each business number, detects conflicting definitions automatically, keeps an unchangeable version history, and lets anyone ask questions in plain English to an assistant called TARA. The key idea is that the AI never produces a figure: it only builds a plan, which is checked against approved metrics and run by a safe formula engine on the real data. The demo is an Indian retailer with rupees, GST, an April-to-March financial year and festival seasons.”

# 3 The 2-minute pitch

> “The problem is metric inconsistency. A Sales team counts the value of goods sold, Finance subtracts returns, and Marketing quotes the invoice amount including GST. All three call it *revenue*. Decisions get made on the wrong number, and generic AI chatbots make it worse because they answer confidently even when they are wrong.
>
> Our project is a **governed semantic layer** with an AI assistant. There are four ideas. First, **one live definition per metric**, enforced by a unique index in the database, with every change stored as an immutable version; rolling back creates a new version instead of rewriting history. Second, **automatic conflict detection**: the system compares formula, aggregation, filters, unit, time window and dataset, rates the severity, and walks the team through assign, review, approve and resolve. On 27 labelled test cases it reached precision 1.0 and recall 1.0, against precision 0.5 for a plain text comparison. Third, **TARA never writes numbers**. A question is turned into a structured plan, validated against the approved catalogue, and compiled by our own parser into parameterised DuckDB SQL; raw SQL is never accepted. Fourth, it is **built for India**: lakh and crore formatting, the financial year starting on 1 April, GST-aware metrics, festival windows such as Diwali, and IST times.

> It is a full stack: React front end, FastAPI back end, PostgreSQL or SQLite, DuckDB on Parquet files, Celery for background jobs, 96 API endpoints, and 325 automated tests. We will also tell you plainly what we did not do: Docker was not available on our machine so the Compose stack was not run, and no live AI key was used, so the AI mode shown is the offline deterministic one."

# 4 Fact sheet

| Item | Fact |
|---|---|
| Project | IntelliMetrics AI; assistant name TARA |
| Team | Devi Priya, Priyanka, Akshay and Bhaskar, KL University, AY 2026-27 |
| Nature | Academic capstone with synthetic data; not a commercial product |
| Size | 46 Python files (7,115 lines), 54 TypeScript files (4,707 lines), 96 endpoints, 20 tables, 25 page components |
| Tests | 325 automated: pytest 136 (also 136 on PostgreSQL 18), Vitest 40, Playwright 13; back-end coverage 82.8 % |
| Demo data | 62,000 synthetic sales lines, 24 columns, Bharat Retail Pvt Ltd; second workspace Sagar Foods & Beverages Ltd |
| Speed (measured) | Metric query engine median 6.4 ms, p95 12.8 ms; ingestion about 86,494 rows per second |
| Not done | Docker never run; no live AI key; no load test; no external security audit; no user study |

# 5 Forty likely questions

## 5.1 About the idea

**1. What problem does your project solve?** Different departments use one name for different calculations, so reports disagree and nobody can say which is right. The project gives each metric one governed definition, detects conflicts automatically and records every change.

**2. Is this not just a BI tool like Power BI or Tableau?** Those tools draw charts; many have a semantic model, but they do not enforce one live definition per metric, detect conflicting definitions across teams, or keep immutable versions with a conflict-resolution workflow. This project focuses on the *governance* layer, with a small dashboard and chat front end to show it working. It is not meant to replace those products and we have not compared against them in a study.

**3. What is a "semantic layer"?** A single place where business terms such as *Revenue* or *Active Customers* are defined once, in terms of the raw data columns, so every report and every question uses the same calculation.

**4. What is new in your work?** The combination: a restricted formula language compiled to parameterised SQL, an automatic multi-factor conflict detector with a severity score and a resolution workflow, immutable versioning enforced in the database, and an AI assistant that is structurally unable to invent numbers. Each idea exists separately in the world; the integrated, tested implementation and its evaluation are the contribution.

**5. Who are the users?** Four roles: Admin, Data Analyst, Reviewer (e.g., a finance controller) and Business User (e.g., a regional sales manager). Each sees and can do only what the permission matrix allows.

**6. Why an Indian context?** To make the demonstration realistic for the audience: rupee formatting with lakh and crore grouping, the April–March financial year, GST, festival seasonality and IST. It also exposes real design questions, for example "last quarter" must mean the last completed *fiscal* quarter, and revenue with and without GST is a genuine conflict.

## 5.2 About AI and TARA

**7. Where exactly does the AI work, and where does it not?** The AI only turns text into a *structured query plan* or a *draft formula*. Numbers come from executing governed metrics on the data. The answer sentence is composed from the returned rows by code. If the AI proposes something not in the approved catalogue, validation rejects it.

**8. How do you prevent hallucination?** Four layers: the planner sees only the authorised catalogue; its JSON output is validated by Pydantic and then by `validate_plan`, which rejects unknown metrics and columns; execution uses the same safe compiler as everything else; and the text is composed from rows. Tests feed forged plans (dimension `ssn`, metric `payroll_secret`) and confirm they are refused.

**9. Which AI model did you use?** The code supports OpenAI and Gemini through their REST APIs, chosen by an environment variable. **We did not have an API key, so no live model was run.** The provider code was tested with mocked responses (timeouts, malformed JSON, hallucinated columns). All results in the report come from the offline deterministic planner.

**10. What is the "offline deterministic mode"?** A rule-based planner that recognises metrics, dimensions, ranking words, time expressions and festivals using synonym tables and regular expressions. It works without internet, is repeatable, and is clearly labelled in the interface. It handles the five suggested Indian questions and variants, and says "we couldn't match that" otherwise.

**11. Isn't a rule-based planner just a toy?** It is limited, and we say so. It is also what makes the tests reproducible. The architecture deliberately separates *planning* (replaceable: rules or an LLM) from *execution and validation* (fixed, safe), so a better planner can be dropped in without changing safety.

**12. What happens when the AI is wrong?** A wrong but valid plan returns a correct number for the wrong question; the answer therefore always shows the metric, version, date range and filters used, so the user can see what was actually asked. An invalid query plan is rejected. A failed AI call falls back to offline mode and says so.

**13. Can TARA modify data or definitions?** No. TARA's generator produces a draft that a human must save and submit; nothing is auto-published or auto-executed, and Ask TARA is read-only.

## 5.3 About the formula language and security

**14. Why did you write your own formula parser?** Letting users or an AI submit SQL is unsafe. A small whitelisted grammar (lexer, recursive-descent parser, typed syntax tree) means only known functions and real columns can appear, and every literal becomes a bound parameter. Injection becomes structurally impossible rather than something to filter.

**15. How do you stop SQL injection?** There is no path for user text to become SQL text: column names are checked against the schema and quoted, functions are whitelisted, and values are `?` parameters. Twenty-five hostile formulas are tested and all are rejected before execution; query-string probes are inert too.

**16. What are the limits on queries?** Formula length 2,000 characters, nesting depth 40, 10,000 returned rows and a 20-second timeout, all configurable. DuckDB runs on read-only views.

**17. How are passwords and sessions handled?** Argon2id password hashes; a 15-minute access token and a rotating 7-day refresh token in httpOnly cookies, so JavaScript cannot read them; refresh tokens are stored hashed and revoked on logout; a custom header protects against cross-site request forgery.

**18. How do you separate companies' data?** Every business table has a workspace id and every query filters by it. Accessing another workspace's object returns 404. A test tries read, write and delete across workspaces for every object type.

**19. Is the system secure enough for production?** It has sound basics (listed above) and tests for them. It has **not** had an external penetration test or dependency scan, the rate limiter is per process, and there is no e-mail provider for password reset. We would call it a secure design for a prototype, not a certified production system.

## 5.4 About conflict detection and governance

**20. How does conflict detection work?** It compares each pair of definitions of the same business idea on six factors: formula (after normalising case, spacing and operand order), aggregation, filters, unit, time window and dataset. Each difference has a type; severity is the maximum, raised when a live metric is involved or several differences combine. A recommendation is produced.

**21. How did you measure accuracy?** Against 27 labelled cases: precision 1.0, recall 1.0. **Caveat: we wrote both the cases and the detector**, so this shows the rules do what we intended, not that they generalise to every company. A text-diff baseline scored precision 0.5 because it flags harmless differences like capitalisation.

**22. Why does your "Revenue" conflict exist?** It is built into the demonstration on purpose: Sales' official Revenue is the taxable value excluding GST; Finance subtracts refunds; Marketing uses the invoice value including GST. Detection finds the Finance and Marketing variants against the official definition.

**23. How do you guarantee one live definition per metric?** A partial unique index in the database, `UNIQUE (workspace, key) WHERE is_live IS TRUE`. Even a bug in the application cannot create two. It was applied on real PostgreSQL 18.

**24. Why are versions immutable, and how is that enforced?** For auditability. An ORM guard raises an error on update or delete, and a database trigger blocks it too, so even a direct SQL statement fails. Rolling back creates a new version that copies an old one.

**25. What does "dashboards follow latest published or pinned" mean?** A dashboard widget can always show the current published definition, or be pinned to a version so its numbers never change silently. It is a trade-off between freshness and stability.

## 5.5 About data and Indian specifics

**26. Where does the data come from?** It is synthetic, generated by `seed_data.py` with a fixed random seed: 62,000 order lines with realistic Indian cities, PIN codes, GST slabs, payment methods, festival-weighted dates and higher returns for cash on delivery. No real person's data is used.

**27. How do you handle the financial year?** Fiscal year starts on 1 April: a date in January belongs to the year that began the previous April. "This year" means financial-year-to-date, and "last quarter" means the last *completed* fiscal quarter (Q1 April–June … Q4 January–March). 14 time phrases are tested against exact dates.

**28. How do you treat GST?** `net_sales` is the taxable value excluding GST and `net_amount` the invoice value including GST. A test checks that invoice value equals taxable value plus GST within a few rupees and that only the official slabs (0, 5, 12, 18, 28 %) appear.

**29. How are lakhs and crores handled?** One formatter in the back end and one in the front end implement Indian digit grouping (12,45,300) and compact forms (₹45 L, ₹1.2 Cr). Both are unit-tested with the same cases.

**30. Why does "Compare Diwali-season sales with the previous quarter" mention "per day"?** The two periods have different lengths, so a raw comparison would be misleading; the answer adds a per-day comparison and says so.

## 5.6 About engineering and results

**31. Why DuckDB and Parquet instead of querying PostgreSQL directly?** Analytical aggregation over tens of thousands of rows is far faster on columnar storage. Measured: the median metric query takes 6.4 ms inside the engine over 62,000 rows. PostgreSQL remains the system of record for metadata, versions and audit.

**32. What are your performance results, and how reliable are they?** Measured with `scripts/benchmark.py` on one Windows machine (22 logical CPUs): ingestion about 86,494 rows per second, metric queries median 6.4 ms, dashboard loads of about 167.2 ms on average. They are single-user numbers; we did not run a load test.

**33. How did you test the system?** 325 automated tests in four suites (pytest, pytest on PostgreSQL, Vitest, Playwright), a coverage report, a benchmark script, and a screenshot sweep of every page in light and dark. Each report row links a test case to its measured result.

**34. What bugs did testing find?** Sixteen are listed in the Testing Report, for example a dashboard table crash when no sort column was set, and a planner that compared the Diwali season with the wrong quarter. Each has a regression test where practical.

**35. Why SQLite and PostgreSQL both?** SQLite gives zero-setup development and fast tests; PostgreSQL is the production target. The same models run on both; migrations were applied on a real PostgreSQL 18.2 server and the whole back-end suite passed there.

**36. Why Celery and Redis?** Large uploads must not block web requests. Celery runs ingestion in a separate worker. Without Redis the API falls back to a background thread so a laptop demo needs nothing extra. We verified a real Celery worker with a real Redis broker.

**37. Did you run Docker? No.** Docker is not installed on our machine, so the Dockerfiles and Compose file are written and reviewed but untested. We verified the services separately (PostgreSQL, Redis with Celery, MySQL as import source). The first Compose run on another machine may reveal a packaging issue.

**38. What are the project's main limitations?** Offline planner is limited; no live AI run; time-saving figures use assumed manual durations; synthetic data only; single-machine measurements; Chromium-only browser tests; no load test, no external security audit, no usability study; no e-mail delivery.

**39. What would you do next?** Run and tune with a real LLM and measure accuracy on a held-out question set; add Redis-backed rate limiting; integrate with real BI sources; add SSO; run a small user study with analysts; and add column-level security.

**40. What did you learn?** That governance problems are mostly about *names and ownership*, not computation; that safe AI integration means constraining the AI's output rather than trusting it; and that measured evidence, including honest limits, is what makes a project defensible.

# 6 Demonstration script (8 minutes)

| Minute | Action | Say |
|---|---|---|
| 0:00 | Landing page, then log in as Reviewer (Meera Iyer) | "University project; Indian retailer with synthetic data." |
| 0:45 | Run conflict detection; open the Finance Revenue conflict | "One word, three definitions. The system found them." |
| 2:00 | Assign, review, approve, resolve with a rename | "Each step recorded; resolving creates a new version." |
| 3:00 | Version history, compare, mention rollback | "Immutable; a rollback is a new version." |
| 3:45 | Log in as Analyst; Formula Lab: `[Revenue] - SUM(refund_amount)` by state; then the injection attempt | "Only whitelisted syntax; values are bound parameters." |
| 5:00 | Log in as Business User; Ask TARA: the Diwali comparison and UPI vs COD | "Rupees, fiscal calendar, sources shown." |
| 6:15 | Ask the weather question | "It refuses instead of guessing." |
| 6:45 | Sales Overview dashboard; Evaluation page | "Numbers measured, not typed." |
| 7:30 | State limitations | "No Docker run, no live AI key, assumptions labelled." |

**Backup plan:** if the network is down the pages still work (photos fall back to gradients). If the database is in a strange state, stop the back end and run `scripts\reset-demo.ps1`; it takes about 2.7 seconds to seed.